TAURUS BIKE SHOP

Section 2 — Chronological Implementation Guide

Complete Step-by-Step Coding Checklist · Every File · 29 Steps

AdminSystem (WPF / .NET 4.8) · WebApplication (ASP.NET Core MVC / .NET 8) · Schema v7.1

How to use this checklist

Work through each step in order. Within a step, files listed in the same sub-group can be implemented in parallel. Each row is a single file with a checkbox column you can tick off as you complete it. Files that have no in-group dependencies are highlighted as parallel in the step band.

STEP 1 Database Schema

The 38 tables, 12 views, and 1 trigger must exist in SQL Server before any C# code can run.

SQL/Schema/

✓	File	What to implement
☐	**Taurus_schema_v7.1.sql**	Run this file against TaurusBikeShopDB. Creates all 38 tables, indexes, 12 views, and TR_Product_PriceAudit trigger. Run once only on a fresh database.
☐	**Taurus_schema_v7.1_patch.sql**	Apply this patch if upgrading from v7.0 to v7.1 on an existing database (adds ReorderThreshold to ProductVariant, creates SupportTask table, fixes vw_OrderSummary GROUP BY).

SQL/Seed/

✓	File	What to implement
☐	**Taurus_seed_v7.1.sql**	Run after schema — inserts 150 products, 43 brands, 13 categories, 150 default variants and images. Required for catalog and POS testing.

STEP 2 WebApplication — Solution & Project Config

Build system and runtime configuration must be correct before any code compiles.

WebApplication/ root

✓	File	What to implement
☐	**WebApplication.slnx**	Canonical solution file. Ensure it references only WebApplication.csproj. Open all development via this file, not OOP-TaurusBikeShop.sln.
☐	**WebApplication.csproj**	Verify: TargetFramework=net8.0 only (remove net7.0/net10.0 if present). Add NuGet packages: EF Core, BCrypt.Net-Next, Google.Cloud.Storage.V1.
☐	**appsettings.json**	Write connection string for TaurusBikeShopDB. Add Session and Logging config. Do not store secrets here — use appsettings.Development.json or secrets manager.
☐	**appsettings.Development.json**	Local dev overrides: local DB connection string, verbose logging. Ensure this file is in .gitignore.

STEP 3 All 38 Entity Classes — Models/Entities/

Every service, repository, AppDbContext, and controller depends on entity classes. All 38 can be written in parallel — they have no inter-dependencies.

Auth & User entities

✓	File	What to implement
☐	**User.cs**	Properties: UserId, Email?, PasswordHash?, FirstName, LastName, PhoneNumber?, DefaultAddressId? (circular FK), IsActive, IsWalkIn, CreatedAt, LastLoginAt?. Nav: Addresses, UserRoles, Orders, Carts, Wishlist, Reviews.
☐	**Role.cs**	Properties: RoleId, RoleName (enum: Admin/Manager/Cashier/Staff/Customer), Description?, CreatedAt. Nav: UserRoles.
☐	**UserRole.cs**	Properties: UserRoleId, UserId, RoleId, AssignedAt. Nav: User, Role. UNIQUE(UserId,RoleId).
☐	**Address.cs**	Properties: AddressId, UserId, Label (enum: Home/Work/Other), Street, City, PostalCode, Province?, Country, IsDefault, IsSnapshot, CreatedAt. Nav: User, Orders.
☐	**OTPVerification.cs**	Properties: OTPId, Email, OTPCode, IsUsed, ExpiresAt, CreatedAt. No FK to User — created before account exists.
☐	**GuestSession.cs**	Properties: GuestSessionId, SessionToken (unique), Email?, PhoneNumber?, ConvertedToUserId?, ExpiresAt, CreatedAt. Nav: Cart?, ConvertedUser?.

Catalog entities

✓	File	What to implement
☐	**Category.cs**	Properties: CategoryId, CategoryCode (unique), Name, Description?, ParentCategoryId? (self-ref), IsActive, DisplayOrder. Nav: Products, ParentCategory, SubCategories.
☐	**Brand.cs**	Properties: BrandId, BrandName (unique), Country?, Website?, Description?, IsActive, CreatedAt. Nav: Products.
☐	**Product.cs**	Properties: ProductId, CategoryId, BrandId?, SKU?, Name, ShortDescription?, Description?, Price, Currency (PHP/USD/EUR), Material?, Color?, WheelSize?, SpeedCompatibility?, BoostCompatible?, TubelessReady?, AxleStandard?, SuspensionTravel?, BrakeType?, AdditionalSpecs?, IsActive, IsFeatured, CreatedAt, UpdatedAt?. Nav: Category, Brand?, Variants, Images, PriceHistory.
☐	**ProductVariant.cs**	Properties: ProductVariantId, ProductId, VariantName, SKU?, AdditionalPrice, StockQuantity, ReorderThreshold (default 5), IsActive, CreatedAt, UpdatedAt?. Nav: Product.
☐	**ProductImage.cs**	Properties: ProductImageId, ProductId, StorageBucket, StoragePath, ImageUrl, ImageType (Full/Medium/Thumbnail), IsPrimary, DisplayOrder, AltText?, FileSizeBytes?, MimeType?, Width?, Height?, UploadedByUserId?, CreatedAt. Nav: Product, UploadedBy?.
☐	**PriceHistory.cs**	Properties: PriceHistoryId, ProductId, OldPrice, NewPrice, ChangedAt, ChangedByUserId?, Notes?. Nav: Product, ChangedBy?. Read-only — written by DB trigger.

Supply chain entities

✓	File	What to implement
☐	**Supplier.cs**	Properties: SupplierId, Name, ContactPerson?, PhoneNumber?, Email? (unique filtered), Address?, IsActive, CreatedAt. Nav: PurchaseOrders.
☐	**PurchaseOrder.cs**	Properties: PurchaseOrderId, SupplierId, OrderDate, ExpectedDeliveryDate?, Status (Pending/Received/Cancelled), CreatedByUserId?, CreatedAt. Nav: Supplier, Items, CreatedBy?.
☐	**PurchaseOrderItem.cs**	Properties: PurchaseOrderItemId, PurchaseOrderId, ProductId, ProductVariantId?, Quantity, UnitPrice. Nav: PurchaseOrder, Product, Variant?. No Subtotal stored — computed.

Voucher entities

✓	File	What to implement
☐	**Voucher.cs**	Properties: VoucherId, Code (unique), Description?, DiscountType (Percentage/Fixed), DiscountValue, MinimumOrderAmount?, MaxUses?, MaxUsesPerUser?, StartDate, EndDate?, IsActive, CreatedAt. Nav: UserVouchers, Usages.
☐	**UserVoucher.cs**	Properties: UserVoucherId, UserId, VoucherId, AssignedAt, ExpiresAt?. Nav: User, Voucher. UNIQUE(UserId,VoucherId).
☐	**VoucherUsage.cs**	Properties: VoucherUsageId, VoucherId, UserId, OrderId, DiscountAmount, UsedAt. Nav: Voucher, User, Order.

Cart & Wishlist entities

✓	File	What to implement
☐	**Cart.cs**	Properties: CartId, UserId?, GuestSessionId?, CreatedAt, LastUpdatedAt?, IsCheckedOut. CHECK: exactly one of UserId/GuestSessionId. Nav: User?, GuestSession?, Items.
☐	**CartItem.cs**	Properties: CartItemId, CartId, ProductId, ProductVariantId?, Quantity, PriceAtAdd, AddedAt. Nav: Cart, Product, Variant?.
☐	**Wishlist.cs**	Properties: WishlistId, UserId, ProductId, AddedAt. Nav: User, Product. UNIQUE(UserId,ProductId).

Order entities

✓	File	What to implement
☐	**Order.cs**	Properties: OrderId, UserId, OrderNumber (unique), OrderDate, OrderStatus (9 values), SubTotal, DiscountAmount, ShippingFee, ShippingAddressId?, ContactPhone?, DeliveryInstructions?, IsWalkIn, CreatedAt. Nav: User, ShippingAddress?, Items, Payments, Delivery?, PickupOrder?.

✓	File	What to implement
☐	**OrderItem.cs**	Properties: OrderItemId, OrderId, ProductId, ProductVariantId?, Quantity, UnitPrice. Nav: Order, Product, Variant?. No Subtotal — computed.
☐	**PickupOrder.cs**	Properties: PickupOrderId, OrderId (unique FK — 1:1), PickupReadyAt?, PickupExpiresAt?, PickupConfirmedAt?. Nav: Order. Configure HasOne/WithOne in AppDbContext.

Inventory entity

✓	File	What to implement
☐	**InventoryLog.cs**	Properties: InventoryLogId, ProductId, ProductVariantId?, ChangeQuantity, ChangeType (8 values), OrderId?, PurchaseOrderId?, ChangedByUserId?, Notes?, CreatedAt. Nav: Product, Variant?, Order?, PurchaseOrder?, ChangedBy?. Immutable — never update or delete rows.

Payment entities (base + 2 subtypes)

✓	File	What to implement
☐	**Payment.cs**	Properties: PaymentId, OrderId, PaymentMethod (GCash/BankTransfer/Cash — NO Card), PaymentStage (Upfront/Confirmation), PaymentStatus (6 values), Amount, PaymentDate?, CreatedAt. Nav: Order, GCashPayment?, BankTransferPayment?.
☐	**GCashPayment.cs**	Properties: PaymentId (PK = FK to Payment — 1:1), GcashTransactionId?. Nav: Payment. Configure HasOne/WithOne/HasForeignKey in AppDbContext.
☐	**BankTransferPayment.cs**	Properties: PaymentId (PK = FK — 1:1), BpiReferenceNumber?, ProofUrl?, ProofStorageBucket?, ProofStoragePath?, VerifiedByUserId?, VerificationNotes?, VerifiedAt?, VerificationDeadline?. Nav: Payment, VerifiedBy?.

Delivery entities (base + 2 subtypes)

✓	File	What to implement
☐	**Delivery.cs**	Properties: DeliveryId, OrderId, Courier (Lalamove/LBC), DeliveryStatus (5 values), IsDelayed, DelayedUntil?, EstimatedDeliveryTime?, ActualDeliveryTime?, CreatedAt. Nav: Order, LalamoveDelivery?, LBCDelivery?.
☐	**LalamoveDelivery.cs**	Properties: DeliveryId (PK = FK — 1:1), BookingRef?, DriverName?, DriverPhone?. Nav: Delivery. Configure HasOne/WithOne in AppDbContext.
☐	**LBCDelivery.cs**	Properties: DeliveryId (PK = FK — 1:1), TrackingNumber?. Nav: Delivery. No driver fields — LBC does not expose via API.

Review & POS entities

✓	File	What to implement
☐	**Review.cs**	Properties: ReviewId, UserId, ProductId, OrderId, Rating (1-5), Comment?, IsVerifiedPurchase, CreatedAt. Nav: User, Product, Order.
☐	**POS_Session.cs**	Properties: POSSessionId, UserId, TerminalName, ShiftStart, ShiftEnd?, TotalSales, CreatedAt. Nav: User.

Support entities

✓	File	What to implement
☐	**SupportTicket.cs**	Properties: TicketId, UserId, OrderId?, TicketSource (Customer/Admin/System), TicketCategory (7 values), Subject, Description?, AttachmentUrl?, AttachmentBucket?, AttachmentPath?, TicketStatus (5 values), AssignedToUserId?, ResolvedAt?, CreatedAt, UpdatedAt?. Nav: User, AssignedTo?, Order?, Replies, Tasks.
☐	**SupportTicketReply.cs**	Properties: ReplyId, TicketId, UserId, IsAdminReply, Message, AttachmentUrl?, AttachmentBucket?, AttachmentPath?, CreatedAt. Nav: Ticket (CASCADE DELETE), User.
☐	**SupportTask.cs**	Properties: TaskId, TicketId, AssignedToUserId?, TaskType (5 values), TaskStatus (4 values), DueDate?, Notes?, CreatedAt, CompletedAt?. Nav: Ticket (CASCADE DELETE), AssignedTo?.

Comms & Audit entities

✓	File	What to implement
☐	**Notification.cs**	Properties: NotificationId, UserId, OrderId?, TicketId?, Channel (Email/SMS), NotifType (17 values), Recipient, Subject?, Body?, Status (Pending/Sent/Failed), RetryCount, SentAt?, FailureReason?, CreatedAt. Nav: User, Order?, Ticket?.
☐	**SystemLog.cs**	Properties: SystemLogId, UserId?, EventType (22 values), EventDescription?, CreatedAt. Nav: User?. UserId NULL for automated events.

STEP 4 AppDbContext — DbSets + Relationship Configuration

Must register all 38 entities and configure the 5 shared-PK 1:1 relationships before EF Core can generate queries.

DataAccess/Context/

✓	File	What to implement
☐	**AppDbContext.cs**	Add DbSet<T> for all 38 entities. In OnModelCreating: configure HasOne/WithOne/HasForeignKey for GCashPayment, BankTransferPayment, LalamoveDelivery, LBCDelivery, PickupOrder (all share PK with parent). Configure UNIQUE indexes for Email, SessionToken, OrderNumber, VoucherCode. Configure self-referencing Category.ParentCategoryId. Configure circular FK User.DefaultAddressId with IsRequired(false).

STEP 5 Utilities

PasswordHelper is needed by UserService before registration can work. All 3 utilities have no dependencies.

Utilities/ — all 3 can be written in parallel

✓	File	What to implement
☐	**PasswordHelper.cs**	Implement Hash(string password): string using BCrypt.Net.BCrypt.HashPassword(password, workFactor:12). Implement Verify(string input, string hash): bool using BCrypt.Net.BCrypt.Verify(input,hash).
☐	**ValidationHelper.cs**	Implement IsValidPhilippinePhone(string phone): bool — accepts +63 or 09 prefix, 11 digits. Implement IsValidPostalCode(string code): bool — 4-digit numeric. Implement IsValidEmail(string email): bool — extend beyond [EmailAddress] attribute.
☐	**FileUploadHelper.cs**	Implement UploadAsync(IFormFile file, string bucket, string folder): Task returning (ImageUrl, StorageBucket, StoragePath). Use Google.Cloud.Storage.V1.StorageClient. Validate MIME type (image/webp, image/jpeg, image/png, application/pdf for proofs). Max 5MB. Return GCS public URL.

STEP 6 Generic Repository — Base Data Access Layer

All specific repositories inherit from Repository.cs. Must exist first.

DataAccess/Repositories/ — write both in parallel

✓	File	What to implement
☐	**IRepository.cs**	Define generic interface IRepository<T>: GetAllAsync(), GetByIdAsync(int id), AddAsync(T entity), UpdateAsync(T entity), DeleteAsync(int id), FindAsync(Expression<Func<T,bool>> predicate).
☐	**Repository.cs**	Implement IRepository<T> using AppDbContext. GetAllAsync: DbSet<T>.ToListAsync(). FindAsync: DbSet<T>.Where(predicate).ToListAsync(). Inject AppDbContext via constructor.

STEP 7 All Specific Repositories

Services depend on repositories. All specific repos can be written in parallel after the base Repository.cs exists.

DataAccess/Repositories/ — all can be written in parallel

✓	File	What to implement
☐	**UserRepository.cs**	Extend Repository<User>. Add: FindByEmailAsync(email), CreateOTPAsync(email, code, expiresAt), VerifyOTPAsync(email, code): checks IsUsed=false and ExpiresAt > now, sets IsUsed=true. UpdateLastLoginAsync(userId).
☐	**BrandRepository.cs**	Extend Repository<Brand>. Add: GetAllActiveBrandsAsync(). Simple — brands are reference data.
☐	**CartRepository.cs**	Extend Repository<Cart>. Add: GetActiveCartAsync(userId? , guestSessionId?): filter IsCheckedOut=false. Include CartItems with Product and Variant. GetCartItemCountAsync(cartId).
☐	**OrderRepository.cs**	Extend Repository<Order>. Add: GetOrdersByUserAsync(userId, pageNum, pageSize). GetOrderWithDetailsAsync(orderId): includes OrderItems, Payments, Delivery, PickupOrder. UpdateStatusAsync(orderId, status). GetPendingUnprocessedAsync(cutoffHours): orders older than cutoff still Pending.
☐	**PaymentRepository.cs**	Extend Repository<Payment>. Add: GetWithSubtypeAsync(paymentId): include GCashPayment and BankTransferPayment. CreateWithSubtypeAsync(payment, subtype): insert Payment row then GCashPayment or BankTransferPayment in same transaction. GetPendingBankTransfersAsync(): STATUS=VerificationPending ordered by CreatedAt.
☐	**ProductRepository.cs**	Extend Repository<Product>. Add: GetFilteredAsync(categoryId?, brandId?, minPrice?, maxPrice?, searchText?, page, pageSize): dynamic EF query with IsActive=true. GetWithFullDetailsAsync(productId): includes Variants, Images (IsPrimary first), Reviews, Brand, Category. GetBySKUAsync(sku).
☐	**ReviewRepository.cs**	Extend Repository<Review>. Add: GetByProductAsync(productId, page, pageSize): ordered by CreatedAt DESC. HasVerifiedPurchaseAsync(userId, productId): checks Order with Delivered status containing productId. GetAverageRatingAsync(productId).
☐	**SupplierRepository.cs**	Extend Repository<Supplier>. Add: GetActiveSuppliersAsync(). GetPurchaseOrdersAsync(supplierId). CreatePurchaseOrderAsync(po, items): inserts PurchaseOrder then all PurchaseOrderItem rows in a transaction. ReceiveStockAsync(purchaseOrderId, receivedItems): updates ProductVariant.StockQuantity and inserts InventoryLog Purchase entries per item. Sets PurchaseOrder.Status=Received.
☐	**SupportRepository.cs**	Extend Repository<SupportTicket>. Add: GetByUserAsync(userId): ordered by CreatedAt DESC. GetWithRepliesAsync(ticketId): include SupportTicketReply ordered by CreatedAt, include SupportTasks. CreateTicketAsync(ticket): insert and return with ID.
☐	**VoucherRepository.cs**	Extend Repository<Voucher>. Add: GetByCodeAsync(code): return Voucher or null. GetGlobalUsageCountAsync(voucherId): COUNT(*) from VoucherUsage. GetUserUsageCountAsync(voucherId, userId): per-user COUNT. WriteUsageAsync(voucherId, userId, orderId, discountAmount): INSERT VoucherUsage.
☐	**WishlistRepository.cs**	Extend Repository<Wishlist>. Add: GetByUserWithProductAsync(userId): join to Product, include primary image. AddIdempotentAsync(userId, productId): catch UniqueConstraint violation silently. GetProductIdsForUserAsync(userId): for stock monitor job. GetUserIdsForProductAsync(productId): for WishlistRestock notification.

STEP 8 Notification Service — Shared Queue Infrastructure

Almost every service that follows will call INotificationService.Queue(). Implement it now as a thin insert so later services can compile without stubs.

BusinessLogic/Interfaces/ + Services/ — write both in parallel

✓	File	What to implement
☐	**INotificationService.cs**	Define: Task QueueAsync(string channel, string notifType, string recipient, string? subject, string? body, int userId, int? orderId=null, int? ticketId=null). Channel: Email or SMS. NotifType: one of 17 CHECK constraint values.
☐	**NotificationService.cs**	Implement INotificationService. QueueAsync: create Notification entity with Status=Pending, RetryCount=0, insert via AppDbContext.SaveChangesAsync(). Does NOT dispatch — dispatch is handled by background jobs polling vw_PendingNotifications.

STEP 9 Program.cs — DI Registrations (initial pass)

Register all services built so far so the app compiles and runs. Will be updated again at Step 19 for background jobs.

WebApplication/ root

✓	File	What to implement
☐	**Program.cs**	Add: builder.Services.AddDbContext<AppDbContext>. AddSession + AddDistributedMemoryCache. AddAuthentication/AddCookie. AddScoped for: IRepository<T>/Repository<T>, all specific repositories (UserRepository etc.), INotificationService/NotificationService. Configure routing: UseAuthentication, UseAuthorization, UseSession. Map controller routes.

STEP 10 Shared Layout + Global Views

_Layout.cshtml, _Navbar, _Footer, _ViewImports, _ViewStart, and Error.cshtml are needed by every page. Implement once here.

Views/ root

✓	File	What to implement
☐	**_ViewImports.cshtml**	Add @using statements for WebApplication.Models.Entities, WebApplication.Models.ViewModels. Add @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers. Add @addTagHelper *, Microsoft.AspNetCore.Mvc.Razor.
☐	**_ViewStart.cshtml**	Set @{ Layout = "~/Views/Shared/_Layout.cshtml"; } — applies to all Customer and Home views.

Views/Shared/

✓	File	What to implement
☐	**_Layout.cshtml**	HTML shell: <head> with meta tags, Bootstrap from ~/lib/bootstrap/dist/css/bootstrap.min.css, site.css, customer.css. <body> with @await Html.PartialAsync("_Navbar"), @RenderBody(), @await Html.PartialAsync("_Footer"). @RenderSection("Scripts",required:false) before </body>. Bootstrap JS from ~/lib/bootstrap/dist/js/bootstrap.bundle.min.js, site.js.
☐	**_Navbar.cshtml**	Top nav: logo, nav links (Home, Catalog, Cart with badge, Login/Logout based on auth state). Cart badge: AJAX call to CartController.GetCartCount returns count. Show/hide auth links based on User.Identity.IsAuthenticated.
☐	**_Footer.cshtml**	Site footer: category quick links, contact info, copyright. Static HTML — no dynamic data.
☐	**_NotificationAlert.cshtml**	Check TempData["success"] and TempData["error"] and TempData["info"]. Render Bootstrap dismissible alert for each. Include auto-dismiss JS.
☐	**_ValidationScriptsPartial.cshtml**	@section Scripts block: render jquery.validate.min.js and jquery.validate.unobtrusive.min.js from ~/lib/. Include in all form views.
☐	**Error.cshtml**	Standard MVC error view. Display RequestId if ShowRequestId=true. No DB queries here.

Views/Home/

✓	File	What to implement
☐	**Index.cshtml**	Hero banner, featured products grid (loop ProductViewModel list from HomeController.Index), category tiles. CTA buttons to ProductCatalog. Add @section Scripts for Index.css.
☐	**Privacy.cshtml**	Static privacy policy page. No model required.

wwwroot/css/ — layout-level CSS (write in parallel)

✓	File	What to implement
☐	**site.css**	Global: CSS custom properties (--primary, --accent, --danger), base typography (font-family: Arial/sans), link styles, utility classes (.text-muted, .btn-primary override).
☐	**responsive.css**	Media queries: @media (max-width:576px), (max-width:768px), (max-width:992px). Mobile-first. Hide sidebar on mobile, stack grids.

✓	File	What to implement
☐	**customer.css**	Shared customer page styles: sidebar, breadcrumbs, page title bars, customer card layout.
☐	**footer.css**	Footer layout: three-column grid, link colours, copyright bar.

wwwroot/js/ — global JS (write in parallel)

✓	File	What to implement
☐	**site.js**	DOMContentLoaded: init Bootstrap tooltips and popovers. Navbar scroll shadow. Global click handlers.
☐	**utils.js**	Export: formatCurrency(amount) returns '₱1,234.00'. debounce(fn, delay). throttle(fn, delay). showAlert(type, message). fetchWithCSRF(url, options): wraps fetch() adding RequestVerificationToken header from meta tag.
☐	**customer.js**	Profile dropdown open/close. Notification bell click handler. Session expiry warning dialog at 5min before timeout.

STEP 11 Authentication — Register, OTP, Login, Profile

Customers must be able to create accounts and log in before any authenticated feature works. Flowchart: Part 1.

ViewModels (write all in parallel)

✓	File	What to implement
☐	**LoginViewModel.cs**	Properties: Email (Required, EmailAddress), Password (Required, DataType.Password). ReturnUrl string for redirect after login.
☐	**RegisterViewModel.cs**	Properties: FirstName, LastName, Email, PhoneNumber, Password, ConfirmPassword (Compare(Password)), Street, City, PostalCode, Province?, Country.
☐	**ProfileViewModel.cs**	Properties: FirstName, LastName, Email (readonly), PhoneNumber?, List<Address> SavedAddresses, Address? NewAddress. ChangePassword sub-model: CurrentPassword, NewPassword, ConfirmNewPassword.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IUserService.cs**	Methods: RegisterAsync(RegisterViewModel vm): send OTP, return success/error. VerifyOTPAsync(email, code): create User + Address. LoginAsync(email, password): return User or null. GetProfileAsync(userId). UpdateProfileAsync(userId, model). ChangePasswordAsync(userId, current, newPwd).
☐	**UserService.cs**	Implement: RegisterAsync calls PasswordHelper.Hash for password, creates OTPVerification, calls INotificationService.Queue(Email, OTPCode...). VerifyOTPAsync: checks OTP, creates User (IsActive=true, IsWalkIn=false), creates Address (IsDefault=true), queues WelcomeEmail. LoginAsync: FindByEmail, BCrypt.Verify, update LastLoginAt.

Controllers/

✓	File	What to implement
☐	**CustomerController.cs**	GET Register → Register.cshtml. POST Register → UserService.RegisterAsync, show _OTPModal on success. POST VerifyOTP → UserService.VerifyOTPAsync, set auth cookie, redirect to Dashboard. GET/POST Login → UserService.LoginAsync, set HttpContext.SignInAsync, redirect. GET Logout → SignOutAsync. GET/POST Profile → UserService.GetProfile/UpdateProfile. GET/POST ChangePassword.
☐	**HomeController.cs**	GET Index → load IProductService.GetFeaturedProducts(count:8), pass as IEnumerable<ProductViewModel> to Index.cshtml. GET Privacy → Privacy.cshtml. GET/POST Error → ErrorViewModel.

Views/Customer/

✓	File	What to implement
☐	**Login.cshtml**	@model LoginViewModel. Form: Email input, Password input, remember-me checkbox, Submit. Link to Register. @Html.AntiForgeryToken(). @section Scripts: _ValidationScriptsPartial.
☐	**Register.cshtml**	@model RegisterViewModel. Multi-field form. On POST success, show _OTPModal.cshtml partial. @section Scripts: ValidationScriptsPartial, otp-modal.css.
☐	**Dashboard.cshtml**	@model DashboardViewModel. Show: recent orders list (last 5), wishlist item count badge, profile name, quick links to OrderHistory/Wishlist/Support. Requires [Authorize].
☐	**Profile.cshtml**	@model ProfileViewModel. Two tabs: Edit Profile (name, phone) and Manage Addresses (list with default star, delete, add new). Change Password section.

Views/Customer/Partials/

✓	File	What to implement
☐	**_OTPModal.cshtml**	Bootstrap modal: 6 individual digit input boxes (or single 6-char input). Resend OTP link (AJAX POST to CustomerController.ResendOTP). Verify button submits form. Countdown timer 10:00 counting down.

wwwroot/css/ (write in parallel with views)

✓	File	What to implement
☐	**auth.css**	Login and register form card styles. Centered card layout. Input group styling. Submit button. Error text in danger color.
☐	**dashboard.css**	Dashboard grid: 4 KPI stat cards, recent orders list, quick link buttons. Responsive collapse.
☐	**profile.css**	Profile info card, editable form sections, address card grid, change password section.
☐	**register.css**	Multi-column registration form. Field grouping. Validation error highlights.

wwwroot/css/ — shared modal CSS

✓	File	What to implement
☐	**otp-modal.css**	OTP modal: digit input boxes (wide, centered, large font). Countdown timer styling. Resend link. Verify button.

STEP 12 Product Catalog — Browse, Search, Filter, Detail

Shopping flow depends on product browsing. Flowchart: Part 2.

ViewModels

✓	File	What to implement
☐	**ProductViewModel.cs**	Properties: ProductId, Name, ShortDescription?, Price (decimal), FormattedPrice (string), Brand?, Category, PrimaryImageUrl, StockQuantity (aggregated), IsInWishlist (bool for heart icon state).
☐	**ProductDetailViewModel.cs**	Properties: Product full model, List<ProductVariant> Variants, List<ProductImage> Images (primary first), List<ReviewViewModel> Reviews, decimal AverageRating, int ReviewCount, bool IsInWishlist, bool CanReview (IsVerifiedPurchase check), int? SelectedVariantId.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IProductService.cs**	Methods: GetFilteredAsync(categoryId?, brandId?, minPrice?, maxPrice?, searchText?, page, pageSize). GetByIdAsync(productId). GetFeaturedAsync(count). GetVariantsAsync(productId). CheckStockAsync(variantId, qty).
☐	**IBrandService.cs**	Methods: GetAllActiveBrandsAsync(). GetByIdAsync(brandId).

✓	File	What to implement
☐	**ProductService.cs**	Implement IProductService using ProductRepository. GetFiltered: build dynamic query. GetById: load full product with variants, images, reviews. CheckStock: query ProductVariant.StockQuantity.
☐	**BrandService.cs**	Implement IBrandService using BrandRepository. Simple pass-through.

Controllers/

✓	File	What to implement
☐	**ProductController.cs**	GET List(categoryId?, brandId?, minPrice?, maxPrice?, search?, page=1): load filtered products, load all brands and categories for sidebar filter, return ProductCatalog.cshtml. GET Detail(productId): load ProductDetailViewModel including variants, images, reviews, IsInWishlist check if authenticated. AJAX GET GetVariantPrice(variantId): return JSON {price, stock} for variant selection.

Views/Customer/

✓	File	What to implement
☐	**ProductCatalog.cshtml**	@model IEnumerable<ProductViewModel>. Filter sidebar: category checkboxes, brand checkboxes, price range sliders. Search input. Sort dropdown. Product grid: loop _ProductCard.cshtml partial. Pagination controls. @section Scripts: product-catalog.js, product-catalog.css.
☐	**ProductDetails.cshtml**	@model ProductDetailViewModel. Image gallery (primary + thumbnails). Spec table. Variant radio buttons (update price+stock via product-catalog.js AJAX). Add-to-cart button (disabled if out of stock). Wishlist heart toggle. Reviews section: star rating display, review list, submit form (if CanReview). @section Scripts: review.js, product-details.css.

Views/Customer/Partials/

✓	File	What to implement
☐	**_ProductCard.cshtml**	@model ProductViewModel. Product thumbnail image, Name (truncated), Brand, formatted Price, stock badge (Out of Stock / Low Stock), Add to Cart button (AJAX), Wishlist heart icon (AJAX toggle).

wwwroot/css/

✓	File	What to implement
☐	**product-catalog.css**	Catalogue layout: two-column (sidebar + grid). Product grid: responsive 2-4 cols. Filter sidebar sticky on desktop. Sort bar.
☐	**product-details.css**	Gallery: main image + thumbnail row. Spec table alternating rows. Variant selector radio cards. Add-to-cart section sticky on mobile.
☐	**product-card.css**	Card: image with hover zoom, title truncation at 2 lines, price bold, stock badge, button group.

wwwroot/js/

✓	File	What to implement
☐	**product-catalog.js**	Filter form: debounce(300ms) search input → AJAX GET ProductController.List with params → replace product grid HTML. Sort dropdown change → reload. Wishlist toggle: AJAX POST WishlistController.Toggle → update heart icon and count badge. Variant price update on ProductDetails: on radio change → AJAX GET GetVariantPrice → update displayed price and stock.

STEP 13 Cart System

Checkout, voucher, and order creation all depend on an active cart. Flowchart: Part 3.

ViewModels

✓	File	What to implement
☐	**CartViewModel.cs**	Properties: List<CartItemViewModel> Items, decimal SubTotal, int ItemCount, string? AppliedVoucherCode, decimal DiscountAmount, decimal GrandTotal. CartItemViewModel: CartItemId, ProductId, VariantId?, ProductName, VariantName?, ImageUrl, Quantity, UnitPrice, LineTotal.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**ICartService.cs**	Methods: GetCartAsync(userId?, guestSessionId?). AddItemAsync(cartOwner, productId, variantId?, qty). UpdateQuantityAsync(cartItemId, newQty). RemoveItemAsync(cartItemId). ClearCartAsync(cartId). MergeGuestCartAsync(guestSessionId, userId): moves items to user cart on login. GetCartCountAsync(userId?, guestSessionId?).
☐	**CartService.cs**	GetCart: call CartRepository.GetActiveCartAsync, create new cart if none exists. AddItem: check stock before adding, update quantity if item already exists. MergeGuestCart: copy guest cart items to user cart, delete guest cart.

Controllers/

✓	File	What to implement
☐	**CartController.cs**	GET ViewCart: load CartViewModel, return Cart.cshtml. AJAX POST AddToCart(productId, variantId?, qty): return JSON {success, cartCount, message}. AJAX POST UpdateQuantity(cartItemId, qty): return JSON {success, lineTotal, subtotal}. AJAX POST RemoveFromCart(cartItemId): return JSON {success, cartCount, subtotal}. AJAX GET GetCartCount: return JSON {count}. All non-view actions return JSON for cart.js.

Views/Customer/

✓	File	What to implement
☐	**Cart.cshtml**	@model CartViewModel. Cart item list (loop _CartItem.cshtml). Order subtotal sidebar. Empty cart message with CTA to shop. Checkout button to CheckoutController. @section Scripts: cart.js, cart.css.

Views/Customer/Partials/

✓	File	What to implement
☐	**_CartItem.cshtml**	@model CartItemViewModel. Product thumbnail, Name, Variant name, Unit price, Quantity stepper (- input +), Line total, Remove button. All interactions via cart.js AJAX. Include data-cartitemid attribute on row.

wwwroot/css/

✓	File	What to implement
☐	**cart.css**	Cart page: two-column layout (item list left, totals sidebar right). Responsive: stack on mobile.
☐	**cart-item.css**	Cart row: flex layout, thumbnail 80px, stepper controls, right-aligned price and remove.

wwwroot/js/

✓	File	What to implement
☐	**cart.js**	Quantity stepper: click +/- → AJAX POST UpdateQuantity → update lineTotal, subtotal, cart badge. Remove: click → AJAX POST RemoveFromCart → remove row with animation, update subtotal and cart badge. Add to cart (from catalog): AJAX POST AddToCart → show toast success, update cart badge count.

STEP 14 Wishlist

Wishlist is independent of checkout but needed for Part 2 shopping flow and Part 12 stock restore notifications. Can be built in parallel with Steps 15-18.

ViewModels

✓	File	What to implement
☐	**WishlistViewModel.cs**	Properties: List<WishlistItemViewModel> Items. WishlistItemViewModel: WishlistId, ProductId, ProductName, PrimaryImageUrl, Price, IsInStock (bool), StockQuantity.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IWishlistService.cs**	Methods: GetWishlistAsync(userId). AddAsync(userId, productId). RemoveAsync(userId, productId). IsInWishlistAsync(userId, productId). GetProductIdsAsync(userId).
☐	**WishlistService.cs**	Implement using WishlistRepository. Add: idempotent, ignore duplicate key exception. GetWishlist: join to Product, include primary image and first variant stock.

Controllers/

✓	File	What to implement
☐	**WishlistController.cs**	GET ViewWishlist: [Authorize], load WishlistViewModel, return Wishlist.cshtml. AJAX POST Toggle(productId): add or remove, return JSON {isInWishlist, wishlistCount}. AJAX POST MoveToCart(productId): remove from wishlist + add to cart.

Views/Customer/

✓	File	What to implement
☐	**Wishlist.cshtml**	@model WishlistViewModel. Product card grid for saved items. In-stock: Add to Cart AJAX button. Out-of-stock: greyed badge. Remove button on each card. @section Scripts: wishlist.js, wishlist.css.

wwwroot/css/ + wwwroot/js/

✓	File	What to implement
☐	**wishlist.css**	Wishlist grid: 3-4 cols responsive. Card: product image, name, price, stock badge, action buttons.
☐	**wishlist.js**	Remove: AJAX POST → remove card with fade, update wishlist count. Move to Cart: AJAX → remove from wishlist grid, update cart badge. Toggle (from catalog): AJAX POST Toggle → swap heart icon filled/outline, update count.

STEP 15 Voucher System

Voucher validation is part of checkout. VoucherController AJAX endpoint must exist before checkout.js can call it. Flowchart: Part 15.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IVoucherService.cs**	Methods: ValidateAsync(code, userId, subtotal): returns VoucherValidationResult {IsValid, ErrorMessage?, DiscountType?, DiscountValue?, DiscountAmount?}. ApplyAsync(voucherId, userId, orderId, discountAmount): writes VoucherUsage row.
☐	**VoucherService.cs**	ValidateAsync: 5 sequential checks — (1) GetByCodeAsync: exists+IsActive. (2) StartDate<=now<=EndDate. (3) subtotal>=MinimumOrderAmount. (4) GetGlobalUsageCount<MaxUses (if set). (5) GetUserUsageCount<MaxUsesPerUser (if set). Calculate DiscountAmount: Percentage=(subtotal*value/100), Fixed=value. Return typed result.

Controllers/

✓	File	What to implement
☐	**VoucherController.cs**	AJAX POST Validate(code, subtotal): [Authorize], call IVoucherService.ValidateAsync, return JSON {valid, discountType, discountAmount, errorMessage}. No view — AJAX only endpoint called by checkout.js.

STEP 16 Checkout + Order Creation

Orchestrates cart → voucher → address → order. Most complex web flow. Flowcharts: Parts 3 and 15.

ViewModels

✓	File	What to implement
☐	**CheckoutViewModel.cs**	Properties: CartViewModel Cart, string? VoucherCode, decimal DiscountAmount, string DeliveryMethod (StorePickup/Lalamove/LBC), string? ShippingAddressLine, string? City, string? PostalCode, string? ContactPhone, string? DeliveryInstructions, decimal ShippingFee, decimal GrandTotal. Validation attributes on address fields when DeliveryMethod != StorePickup.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IOrderService.cs**	Methods: CreateOrderAsync(userId, cartId, checkoutVm, voucherId?): atomic transaction. GetOrdersByUserAsync(userId, page, pageSize). GetOrderByIdAsync(orderId). UpdateStatusAsync(orderId, status). CancelOrderAsync(orderId, userId).
☐	**OrderService.cs**	CreateOrderAsync: DB transaction — (1) copy CartItems to OrderItems with UnitPrice snapshot, (2) write InventoryLog Lock entries per variant, (3) insert Order (Status=Pending for GCash/BankTransfer, Processing for Cash/POS), (4) clear cart (IsCheckedOut=true), (5) if voucher: call IVoucherService.ApplyAsync, (6) queue OrderConfirmation notification. Rollback on any failure.

Controllers/

✓	File	What to implement
☐	**CheckoutController.cs**	GET Review: [Authorize], load cart into CheckoutViewModel. GET Address: show address form. POST Address: validate, create Address snapshot (IsSnapshot=true). GET Delivery: show _DeliverySelection partial. POST Delivery: set DeliveryMethod and ShippingFee. GET Payment: redirect to PaymentController. POST Confirm: [ValidateAntiForgeryToken], call IOrderService.CreateOrderAsync, redirect to OrderController.Confirmation(orderId).

Views/Customer/

✓	File	What to implement
☐	**Checkout.cshtml**	@model CheckoutViewModel. Multi-step layout: Order Review → Address → Delivery → Payment. Order summary sidebar (_OrderSummary.cshtml). Voucher code field with Apply button. Delivery selection (_DeliverySelection.cshtml). @section Scripts: checkout.js, checkout.css.

Views/Customer/Partials/

✓	File	What to implement
☐	**_OrderSummary.cshtml**	@model CheckoutViewModel (or CartViewModel). Line items list, subtotal, discount row (shown only if DiscountAmount > 0), shipping fee row, grand total bold. Updates dynamically via checkout.js.
☐	**_DeliverySelection.cshtml**	Three options: Store Pickup (free), Lalamove (show estimated fee), LBC (show estimated fee). Radio group. Selecting option updates ShippingFee via checkout.js.

wwwroot/css/ + wwwroot/js/

✓	File	What to implement
☐	**checkout.css**	Multi-step checkout: step indicator bar, form sections, address card, delivery radio options, order summary sidebar sticky.
☐	**checkout.js**	Voucher code: on blur → fetchWithCSRF POST to VoucherController.Validate → if valid: show discount line in _OrderSummary, update GrandTotal; if invalid: show error message. Delivery method change: update ShippingFee in summary. Form step navigation: show/hide sections.

STEP 17 Payment Processing

Payment comes after order creation. Flowchart: Part 4.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IPaymentService.cs**	Methods: InitiateGCashAsync(orderId, amount). InitiateBankTransferAsync(orderId, amount). RecordCashPaymentAsync(orderId, amount). GetPaymentStatusAsync(paymentId). UploadBankProofAsync(paymentId, file). NOTE: NO InitiateCard method — card is not offered.
☐	**PaymentService.cs**	InitiateGCash: create Payment(GCash, Pending), create GCashPayment row, call external GCash gateway API, store transaction ref, update status. InitiateBankTransfer: create Payment(BankTransfer, Pending), create BankTransferPayment row with VerificationDeadline=now+24h, queue PaymentHeld notification. RecordCash: POS only — create Payment(Cash, Completed), queue PaymentReceived. UploadBankProof: call FileUploadHelper.UploadAsync, update BankTransferPayment.ProofUrl/Bucket/Path, set Status=VerificationPending, queue PaymentReceived notification.

Controllers/

✓	File	What to implement
☐	**PaymentController.cs**	GET Index(orderId): load order, show Payment.cshtml with method selection. POST SubmitGCash(orderId, referenceNumber): call IPaymentService.InitiateGCash, redirect to tracking. POST SubmitBankTransfer(orderId, file): call IPaymentService.InitiateBankTransfer + UploadBankProof, redirect to confirmation. POST SubmitCash(orderId): POS admin only (AdminSystem handles this — web should not expose cash endpoint). AJAX GET GetStatus(paymentId): return JSON for payment.js polling.

Views/Customer/

✓	File	What to implement
☐	**Payment.cshtml**	@model Order (with Payment info). Method selector: GCash section (QR code placeholder, reference input), BankTransfer section (bank account details, file upload for proof). NO Card option. Submit button. @section Scripts: payment.js, payment.css.

Views/Customer/Partials/

✓	File	What to implement
☐	**_PaymentMethod.cshtml**	Radio cards for GCash and BankTransfer. GCash card: QR image, reference number text input. BankTransfer card: bank account details (BDO/BPI info), file upload input for proof screenshot. Conditional show/hide per selection. NO Card radio.
☐	**_OTPModal.cshtml**	Already created in Step 11 — used here for GCash OTP verification step if gateway requires it.

wwwroot/css/ + wwwroot/js/

✓	File	What to implement
☐	**payment.css**	Payment page: method card layout, QR placeholder, file upload drop zone, submit button.
☐	**payment-method.css**	Payment method radio cards: icon, label, selected border highlight, conditional content sections.
☐	**payment.js**	Method toggle: show/hide GCash vs BankTransfer sections based on radio selection. File upload preview: show filename on selection. AJAX GET GetStatus polling (every 3s for GCash pending state). Stop polling on Completed/Failed result.

STEP 18 Order Confirmation + Tracking + History

Post-payment pages. Flowchart: Part 5.

ViewModels

✓	File	What to implement
☐	**OrderViewModel.cs**	Properties: OrderId, OrderNumber, OrderDate (formatted), OrderStatus, StatusBadgeClass (CSS class string), SubTotal, DiscountAmount, ShippingFee, GrandTotal (computed), List<OrderItemViewModel> Items, DeliveryStatus?, TrackingNumber?, PickupExpiresAt? (formatted), IsWalkIn. OrderItemViewModel: ProductName, VariantName?, Quantity, UnitPrice, LineTotal, ImageUrl.

Controllers/

✓	File	What to implement
☐	**OrderController.cs**	GET Confirmation(orderId): [Authorize], load order with full details, return Confirmation.cshtml. GET History(page=1): [Authorize], load paginated orders for userId, return OrderHistory.cshtml. GET Detail(orderId): [Authorize], verify order belongs to userId, return order detail JSON or partial. GET Track(orderId): return delivery status JSON for order-history.js polling.

Views/Customer/

✓	File	What to implement
☐	**Confirmation.cshtml**	@model OrderViewModel. Order success banner with confetti (confirmation.js). Order number prominently displayed. Order items table. Payment method used. Delivery/pickup info. CTAs: Track Order, Continue Shopping. @section Scripts: confirmation.js, confirmation.css.
☐	**OrderHistory.cshtml**	@model IEnumerable<OrderViewModel>. Order cards list: OrderNumber, date, status badge (colour-coded), total, View Details link, Track button. Status filter tabs (All/Pending/Processing/Delivered/Cancelled). @section Scripts: order-history.js, order-history.css.

Views/Customer/Partials/

✓	File	What to implement
☐	**_ConfirmationModal.cshtml**	Bootstrap modal auto-shown on Confirmation.cshtml load. Success icon, order number, estimated delivery message. Button: View Order History.

wwwroot/css/ + wwwroot/js/

✓	File	What to implement
☐	**confirmation.css**	Success banner: green gradient, checkmark icon. Order detail card. Next steps section.
☐	**confirmation-modal.css**	Modal: success animation, order number large, CTA button.
☐	**order-history.css**	Order card: status badge colour map, expandable detail row.
☐	**order-history.js**	Status filter tabs: show/hide cards client-side. Expand/collapse: toggle order items detail row. Track polling: AJAX GET Track(orderId) every 10s for active deliveries, update status text without page reload.

STEP 19 Reviews

Post-delivery review submission. Only available after order is Delivered. Flowchart: Part 6.

ViewModels

✓	File	What to implement
☐	**ReviewViewModel.cs**	Properties: ReviewId?, ProductId, ProductName, OrderId, Rating (1-5), Comment?, IsVerifiedPurchase, ReviewerName, CreatedAt (formatted). SubmitReviewViewModel: ProductId, OrderId, Rating (Range(1,5)), Comment?.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**IReviewService.cs**	Methods: GetByProductAsync(productId, page, pageSize). SubmitReviewAsync(userId, productId, orderId, rating, comment?). HasVerifiedPurchaseAsync(userId, productId). GetAverageRatingAsync(productId).
☐	**ReviewService.cs**	SubmitReview: call HasVerifiedPurchase, throw if not. Insert Review with IsVerifiedPurchase=true. Queue ReviewConfirmation notification. GetByProduct: paginated, ordered by CreatedAt DESC.

Controllers/

✓	File	What to implement
☐	**ReviewController.cs**	AJAX GET GetProductReviews(productId, page): return JSON list for ProductDetails page lazy load. POST SubmitReview(model): [Authorize], call IReviewService.SubmitReview, return JSON {success, newReviewHtml} for AJAX append.

Views/Customer/

✓	File	What to implement
☐	**Reviews.cshtml**	@model IEnumerable<ReviewViewModel>. Star rating display (CSS-only stars based on Rating). Review cards: reviewer name, date, verified badge, comment. Submit form at top (if CanReview): star-rating widget (5 clickable stars), comment textarea. @section Scripts: review.js, review.css.

wwwroot/css/ + wwwroot/js/

✓	File	What to implement
☐	**review.css**	Star rating: CSS stars (filled/empty/half). Review card: verified badge green. Submit form: star widget clickable states.
☐	**review.js**	Star widget: click to set rating, hover preview. SubmitReview: AJAX POST → if success: prepend new review card, clear form, show success toast. GetProductReviews: lazy load on ProductDetails scroll.

STEP 20 Support Tickets (Web)

Customers create tickets from Dashboard (Part 2) and Delivery (Part 6). Flowchart: Part 13.

BusinessLogic/Interfaces/ + Services/

✓	File	What to implement
☐	**ISupportService.cs**	Methods: CreateTicketAsync(userId, orderId?, category, subject, description?, attachmentFile?). GetByUserAsync(userId). GetByIdAsync(ticketId).
☐	**SupportService.cs**	CreateTicket: if attachmentFile provided, call FileUploadHelper.UploadAsync, set AttachmentUrl/Bucket/Path. Insert SupportTicket with Status=Open, Source=Customer. Queue SupportTicketCreated notification to user and admin.

Controllers/

✓	File	What to implement
☐	**SupportController.cs**	POST Create(category, subject, description?, orderId?, file?): [Authorize], call ISupportService.CreateTicketAsync, redirect to Dashboard with success TempData. GET MyTickets: [Authorize], load GetByUserAsync, return partial or JSON for Dashboard widget.

STEP 21 DashboardViewModel + ViewModels Completion

Customer dashboard needs aggregated data from orders, wishlist, and notifications. Build the ViewModel and wire to CustomerController.Dashboard.

ViewModels

✓	File	What to implement
☐	**DashboardViewModel.cs**	Properties: string CustomerName, List<OrderViewModel> RecentOrders (last 5), int WishlistCount, int UnreadNotifications, List<SupportTicket> OpenTickets. Populated by CustomerController pulling from IOrderService, IWishlistService, SupportService.
☐	**ErrorViewModel.cs**	Properties: string? RequestId, bool ShowRequestId. Already scaffolded by ASP.NET template — verify it exists and has correct namespace.

STEP 22 Background Jobs — All 5 IHostedService Workers

Automated monitoring for payment timeouts, stock levels, delivery tracking, pending orders. All 5 can be written in parallel. Flowchart: Part 14.

BackgroundJobs/ — all 5 can be written in parallel

✓	File	What to implement
☐	**InventorySyncJob.cs**	Implement BackgroundService. ExecuteAsync: while not cancelled, sleep 10 seconds. On each tick: query ProductVariant where StockQuantity changed since last sync timestamp (use SystemLog or timestamp column). Write unified InventoryLog entry with ChangeType=Adjustment if discrepancy detected. Log BackgroundJobStart/Complete/Error events to SystemLog.
☐	**PendingOrderMonitorJob.cs**	Every 5 minutes: query Orders where Status=Pending AND CreatedAt < now-24h. For each: queue PendingOrderAlert notification to admin. Write PendingOrderAlert to SystemLog. Only alert once: check if PendingOrderAlert Notification already exists for orderId.
☐	**PaymentTimeoutJob.cs**	Every 5 minutes: query BankTransferPayment JOIN Payment where VerificationDeadline < now AND PaymentStatus=VerificationPending. For each (in a DB transaction): set Order.Status=OnHold, set Payment.PaymentStatus=Failed, queue PaymentHeld notification to customer. Log PaymentTimeout to SystemLog.
☐	**StockMonitorJob.cs**	Every 15 minutes: query ProductVariant where StockQuantity < ReorderThreshold. For each: queue LowStockAlert to admin. Call WishlistRepository.GetUserIdsForProductAsync(productId) to find wishlist customers — queue WishlistRestock notification for each. Avoid duplicate alerts: check Notification table for existing unsent LowStockAlert per productId.
☐	**DeliveryStatusPollJob.cs**	Every 5 minutes: query LalamoveDelivery JOIN Delivery where DeliveryStatus NOT IN (Delivered, Failed). Call Lalamove API per booking ref, update DeliveryStatus. Same for LBCDelivery. On status change: queue TrackingUpdate notification. On Delivered: update Order.Status=Delivered, write InventoryLog Unlock entries. Log DeliveryStatusPoll to SystemLog. Handle API failures gracefully.

Program.cs — add background job registrations

✓	File	What to implement
☐	**Program.cs**	Add to DI registrations: builder.Services.AddHostedService<InventorySyncJob>(). AddHostedService<PendingOrderMonitorJob>(). AddHostedService<PaymentTimeoutJob>(). AddHostedService<StockMonitorJob>(). AddHostedService<DeliveryStatusPollJob>(). Also add scoped services needed by jobs: IBrandService, IWishlistService registrations if not already added.

STEP 23 Supplier & Purchase Order (Web side — read-only)

SupplierRepository feeds the AdminSystem for stock receiving. Web side has entities and repository only — no controller or views. Flowchart: Part 16.

DataAccess/Repositories/

✓	File	What to implement
☐	**SupplierRepository.cs**	Already specified in Step 7. Verify GetActiveSuppliersAsync(), GetPurchaseOrdersAsync(supplierId), CreatePurchaseOrderAsync(), and ReceiveStockAsync() are all implemented. ReceiveStockAsync is the most critical — used by AdminSystem to update stock and write InventoryLog Purchase entries.

STEP 24 AdminSystem — Foundation: Config, Helpers, Repositories

All AdminSystem services depend on DatabaseHelper. Repositories depend on DatabaseHelper and IRepository. All repo files can be written in parallel after DatabaseHelper exists.

AdminSystem root config

✓	File	What to implement
☐	**AdminSystem.csproj**	Verify: .NET Framework 4.8 target. Add NuGet packages: Dapper, BCrypt.Net-Next, System.Data.SqlClient (or Microsoft.Data.SqlClient).
☐	**App.config**	Add connection string: <connectionStrings><add name="TaurusBikeShopDB" connectionString="Server=...;Database=TaurusBikeShopDB;..." /></connectionStrings>. Match the same TaurusBikeShopDB that WebApplication uses.
☐	**App.xaml**	WPF application entry point. In <Application.Resources>: merge AppStyles.xaml resource dictionary.
☐	**App.xaml.cs**	Application_Startup: create DI container (use Simple Injector or manual constructor injection). Register all services and ViewModels. Set startup window to new LoginWindow().
☐	**AppStyles.xaml**	Define: ButtonStyle, TextBoxStyle, DataGridStyle. Named brushes: PrimaryBrush (#1F3864), AccentBrush (#00695C), DangerBrush (#B71C1C), WarningBrush (#F57F17). Font size definitions: SmallFont(12), NormalFont(14), LargeFont(16).

AdminSystem Helpers/ (write in parallel)

✓	File	What to implement
☐	**DatabaseHelper.cs**	Implement GetConnection(): SqlConnection — read connection string from ConfigurationManager.ConnectionStrings["TaurusBikeShopDB"].ConnectionString. Open connection, return it. All repositories call this.
☐	**NavigationHelper.cs**	Implement NavigateTo<TView>(FrameworkElement contentFrame, object? dataContext): creates new instance of TView, sets DataContext, loads into contentFrame. Store CurrentView for back navigation.
☐	**PasswordHelper.cs**	Same as WebApplication: Verify(input, hash) using BCrypt.Net.BCrypt.Verify.
☐	**PrintHelper.cs**	Implement GenerateReceipt(Order order, List<OrderItem> items, decimal total): build FlowDocument with shop header, itemised table, total, payment method. Call PrintDialog.PrintDocument().

AdminSystem Converters/ (write in parallel)

✓	File	What to implement
☐	**BoolToVisibility.cs**	IValueConverter: Convert(bool) → Visibility.Visible or Visibility.Collapsed. ConvertBack: Visibility → bool. Register in AppStyles.xaml as StaticResource.
☐	**CurrencyConverter.cs**	IValueConverter: Convert(decimal) → string formatted as '₱#,##0.00' using Philippine culture. Register in AppStyles.xaml.
☐	**StatusToColor.cs**	IValueConverter: Convert(string status) → SolidColorBrush. Map: Pending=#FFA500, Processing=#2196F3, ReadyForPickup=#9C27B0, Delivered=#4CAF50, Cancelled=#F44336, OnHold=#FF5722, Shipped=#03A9F4.

AdminSystem Models/ (write in parallel — all 18 POCOs)

✓	File	What to implement
☐	**AdminSystem/Brand.cs**	Properties: BrandId, BrandName, Country, Website, IsActive.
☐	**AdminSystem/Category.cs**	Properties: CategoryId, CategoryCode, Name, ParentCategoryId?, IsActive, DisplayOrder.
☐	**AdminSystem/Delivery.cs**	Properties: DeliveryId, OrderId, Courier, DeliveryStatus, IsDelayed, DelayedUntil?, DriverName?, DriverPhone?, BookingRef?, TrackingNumber?, EstimatedDeliveryTime?, ActualDeliveryTime?, CreatedAt.
☐	**AdminSystem/GCashPayment.cs**	Properties: PaymentId, GcashTransactionId?.
☐	**AdminSystem/BankTransferPayment.cs**	Properties: PaymentId, BpiReferenceNumber?, ProofUrl?, VerifiedByUserId?, VerificationNotes?, VerifiedAt?, VerificationDeadline?.

✓	File	What to implement
☐	**AdminSystem/InventoryLog.cs**	Properties: InventoryLogId, ProductId, ProductVariantId?, ChangeQuantity, ChangeType, OrderId?, PurchaseOrderId?, ChangedByUserId?, Notes?, CreatedAt.
☐	**AdminSystem/LalamoveDelivery.cs**	Properties: DeliveryId, BookingRef?, DriverName?, DriverPhone?.
☐	**AdminSystem/LBCDelivery.cs**	Properties: DeliveryId, TrackingNumber?.
☐	**AdminSystem/Order.cs**	Properties: OrderId, UserId, OrderNumber, OrderDate, OrderStatus, SubTotal, DiscountAmount, ShippingFee, IsWalkIn, CreatedAt. Include sub-objects: List<OrderItem> Items, PickupOrder? Pickup.
☐	**AdminSystem/OrderItem.cs**	Properties: OrderItemId, OrderId, ProductId, ProductVariantId?, ProductName (join), VariantName? (join), Quantity, UnitPrice.
☐	**AdminSystem/Payment.cs**	Properties: PaymentId, OrderId, PaymentMethod (GCash/BankTransfer/Cash — NO Card), PaymentStatus, Amount, PaymentDate?, CreatedAt.
☐	**AdminSystem/PickupOrder.cs**	Properties: PickupOrderId, OrderId, PickupReadyAt?, PickupExpiresAt?, PickupConfirmedAt?.
☐	**AdminSystem/POS_Session.cs**	Properties: POSSessionId, UserId, TerminalName, ShiftStart, ShiftEnd?, TotalSales, CreatedAt.
☐	**AdminSystem/Product.cs**	Properties: ProductId, CategoryId, BrandId?, SKU?, Name, Price, Material?, Color?, WheelSize?, SpeedCompatibility?, IsActive, IsFeatured, CreatedAt. List<ProductVariant> Variants.
☐	**AdminSystem/Review.cs**	Properties: ReviewId, UserId, ProductId, OrderId, Rating, Comment?, IsVerifiedPurchase, CreatedAt.
☐	**AdminSystem/SupportTask.cs**	Properties: TaskId, TicketId, AssignedToUserId?, TaskType, TaskStatus, DueDate?, Notes?, CreatedAt, CompletedAt?.
☐	**AdminSystem/User.cs**	Properties: UserId, Email?, FirstName, LastName, PhoneNumber?, IsActive, IsWalkIn, CreatedAt. Role string (from UserRole join).
☐	**AdminSystem/Voucher.cs**	Properties: VoucherId, Code, DiscountType, DiscountValue, MinimumOrderAmount?, MaxUses?, MaxUsesPerUser?, StartDate, EndDate?, IsActive.

AdminSystem Repositories/ (write in parallel after DatabaseHelper)

✓	File	What to implement
☐	**AdminSystem/IRepository.cs**	Generic interface: GetAll<T>(), GetById<T>(int id), Insert<T>(T entity), Update<T>(T entity), Delete<T>(int id). Uses Dapper.
☐	**AdminSystem/Repository.cs**	Generic Dapper implementation. GetAll<T>: connection.QueryAsync<T>("SELECT * FROM [TableName]"). Use Dapper with T attribute or explicit SQL per repo.
☐	**AdminSystem/InventoryRepository.cs**	GetStockLevels(): JOIN Product+ProductVariant with ReorderThreshold. GetInventoryLog(productId?): InventoryLog entries. AdjustStock(productId, variantId, changeQty, changeType, changedByUserId, notes): UPDATE ProductVariant.StockQuantity, INSERT InventoryLog. Use SqlTransaction.
☐	**AdminSystem/OrderRepository.cs**	GetPendingOrders(): Status IN (Pending, PendingVerification, Processing). GetOrderWithDetails(orderId): JOIN OrderItem+Product+Variant. UpdateOrderStatus(orderId, status). SetPickupReady(orderId): sets PickupReadyAt=now, PickupExpiresAt=now+3days. ConfirmPickup(orderId): sets PickupConfirmedAt=now, Status=PickedUp.
☐	**AdminSystem/PaymentRepository.cs**	GetPendingBankTransfers(): JOIN Payment+BankTransferPayment WHERE Status=VerificationPending, ordered by CreatedAt. ApprovePayment(paymentId, adminUserId, notes): sets VerifiedAt, VerifiedByUserId, Status=Completed. RejectPayment(paymentId, adminUserId, reason): sets VerificationNotes, Status=VerificationRejected. RecordPOSPayment(orderId, method, amount): INSERT Payment + GCashPayment or no subtype for Cash.
☐	**AdminSystem/ProductRepository.cs**	GetBySKU(sku): fast lookup for POS barcode scan. GetWithVariants(productId): JOIN ProductVariant. UpdateProduct(product). CreateProduct(product, variants). GetLowStock(): WHERE StockQuantity < ReorderThreshold.
☐	**AdminSystem/UserRepository.cs**	FindByEmail(email): return User with role name from UserRole JOIN. GetAllStaff(): WHERE IsWalkIn=false AND UserRole has Admin/Manager/Cashier/Staff. UpdateIsActive(userId, isActive). AssignRole(userId, roleId). RemoveRole(userId, roleId).

STEP 25 AdminSystem — All Services and Interfaces

All ViewModels call services. All service files can be written in parallel after their repository dependencies exist.

AdminSystem Services/ — all can be written in parallel

✓	File	What to implement
☐	**IAuthService.cs / AuthService.cs**	IAuthService: Login(email, password): User?, ValidateSession(token): bool. AuthService: FindByEmail, BCrypt.Verify, check role is Admin/Manager/Cashier/Staff, return User with role. Store session token in memory (Dictionary or App-level).
☐	**IDeliveryService.cs / DeliveryService.cs**	IDelivery: GetPendingDeliveries(), DispatchOrder(orderId, courier), MarkDelayed(deliveryId, newETA), CancelDelivery(deliveryId). DeliveryService: DispatchOrder calls Lalamove or LBC external API, stores BookingRef/TrackingNumber, sets Status=Shipped, calls INotificationService for TrackingUpdate.
☐	**IInventoryService.cs / InventoryService.cs**	IInventory: AdjustStock(productId, variantId?, qty, changeType, notes?), GetLowStockItems(), GetInventoryLog(productId?). InventoryService: AdjustStock calls InventoryRepository.AdjustStock, then checks if StockQuantity < ReorderThreshold → call INotificationService for LowStockAlert.
☐	**AdminSystem/INotificationService.cs / NotificationService.cs**	Same interface as WebApplication version. NotificationService: INSERT Notification row into shared DB via DatabaseHelper (direct SQL, not EF). This writes to the same Notification table that the WebApplication background job processes.
☐	**IOrderService.cs / OrderService.cs**	IOrder: GetOrders(status?, page), GetOrderWithDetails(orderId), UpdateOrderStatus(orderId, status), CreateWalkInOrder(items, paymentMethod, cashierId, sessionId), SetPickupReady(orderId). CreateWalkInOrder: DB transaction — INSERT Order(IsWalkIn=true, Status=Delivered instantly), INSERT OrderItems, UPDATE ProductVariant StockQuantity, INSERT InventoryLog Sale, INSERT Payment(Cash or GCash), INSERT POS_Session update.
☐	**IPaymentService.cs / PaymentService.cs**	IPayment: ProcessCash(orderId, amount, sessionId), ProcessGCash(orderId, amount), RecordPayment(paymentData). NO ProcessCard. PaymentService: ProcessCash → INSERT Payment(Cash, Completed) + no subtype. ProcessGCash → INSERT Payment(GCash, Pending) + INSERT GCashPayment, call external GCash API.
☐	**IProductService.cs / ProductService.cs**	IProduct: SearchBySKU(sku), SearchByName(term), GetWithVariants(productId), UpdateProduct(product), CreateProduct(product). ProductService: SearchBySKU used for POS barcode scan — returns product + all variants + stock levels.
☐	**IReportService.cs / ReportService.cs**	IReport: GetSalesSummary(fromDate, toDate): total revenue, order count, avg order value. GetTopProducts(fromDate, toDate, count): best sellers by revenue. GetInventoryReport(): all products with stock status vs ReorderThreshold. GetRevenueByDate(fromDate, toDate): daily/weekly aggregation. ReportService: raw Dapper queries across Order, OrderItem, Product, InventoryLog tables.
☐	**ISupportService.cs / SupportService.cs**	ISupport: GetOpenTickets(status?), GetTicketWithReplies(ticketId), ReplyToTicket(ticketId, adminUserId, message, attachmentFile?), CreateTask(ticketId, taskType, assignedTo?, dueDate?, notes?), ResolveTicket(ticketId). SupportService: ReplyToTicket → INSERT SupportTicketReply(IsAdminReply=true), UPDATE SupportTicket.TicketStatus=AwaitingResponse, call INotificationService for SupportTicketReply. ResolveTicket: SET ResolvedAt=now, Status=Resolved, queue SupportTicketResolved notification.

STEP 26 AdminSystem — ViewModels (BaseViewModel first, then all others in parallel)

All ViewModels inherit BaseViewModel. Write BaseViewModel first, then all 11 feature ViewModels can be done simultaneously.

AdminSystem ViewModels/

✓	File	What to implement
☐	**BaseViewModel.cs**	Implement INotifyPropertyChanged. protected void SetProperty<T>(ref T field, T value, [CallerMemberName] string? name=null): compare, set field, raise PropertyChanged(name). All VMs inherit this.
☐	**LoginViewModel.cs**	Properties: Email, Password, ErrorMessage, IsLoading. LoginCommand (ICommand): validate fields, call AuthService.Login, if success call NavigationHelper.NavigateTo<MainWindow>, else set ErrorMessage.

✓	File	What to implement
☐	**DashboardViewModel.cs**	Properties: decimal TodaySales, int PendingOrders, int LowStockCount, int OpenTickets. LoadCommand: call ReportService.GetSalesSummary(today), OrderRepository.GetPendingOrders().Count, ProductRepository.GetLowStock().Count, SupportRepository.GetOpenTickets().Count.
☐	**OrderViewModel.cs**	Properties: ObservableCollection<Order> Orders, Order? SelectedOrder, string StatusFilter, List<OrderItem> SelectedOrderItems. LoadCommand, StatusUpdateCommand (calls OrderService.UpdateOrderStatus), SetPickupReadyCommand (calls OrderService.SetPickupReady), ConfirmPickupCommand.
☐	**POSViewModel.cs**	Properties: string SearchText, ObservableCollection<OrderItem> CartItems, decimal SubTotal, decimal GrandTotal, string PaymentMethod, string? VoucherCode, decimal Discount, POS_Session? CurrentSession. SearchCommand (barcode/SKU lookup), AddToCartCommand, RemoveFromCartCommand, ApplyVoucherCommand, CompleteTransactionCommand (calls OrderService.CreateWalkInOrder, PaymentService, PrintHelper), OpenSessionCommand, CloseSessionCommand.
☐	**InventoryViewModel.cs**	Properties: ObservableCollection<Product> Products, Product? SelectedProduct, int AdjustQuantity, string AdjustType, ObservableCollection<InventoryLog> AuditLog. AdjustStockCommand (calls InventoryService.AdjustStock), LoadLowStockCommand, LoadAuditLogCommand.
☐	**ProductViewModel.cs**	Properties: ObservableCollection<Product> Products, Product EditingProduct, ObservableCollection<Brand> Brands, ObservableCollection<Category> Categories. SaveCommand (create or update), DeleteCommand, AddVariantCommand.
☐	**DeliveryViewModel.cs**	Properties: ObservableCollection<Delivery> PendingDeliveries, Delivery? SelectedDelivery. DispatchCommand (calls DeliveryService.DispatchOrder), MarkDelayedCommand (prompt for new ETA), CancelDeliveryCommand.
☐	**ReportViewModel.cs**	Properties: DateTime FromDate, DateTime ToDate, string ReportType, DataTable ReportData, decimal TotalRevenue, int OrderCount. GenerateReportCommand (calls ReportService), ExportCommand (StreamWriter CSV).
☐	**SupportViewModel.cs**	Properties: ObservableCollection<SupportTicket> Tickets, SupportTicket? SelectedTicket, string ReplyText, ObservableCollection<SupportTask> Tasks. ReplyCommand (calls SupportService.ReplyToTicket), CreateTaskCommand (show task form), ResolveCommand (calls SupportService.ResolveTicket). StatusFilter ComboBox.
☐	**UsersViewModel.cs**	Properties: ObservableCollection<User> Users, User? SelectedUser. ToggleActiveCommand (calls UserRepository.UpdateIsActive), AssignRoleCommand, RemoveRoleCommand. LoadAll on init.
☐	**VoucherViewModel.cs**	Properties: ObservableCollection<Voucher> Vouchers, Voucher EditingVoucher. CreateCommand (validate Code uniqueness, dates), ToggleActiveCommand.

STEP 27 AdminSystem — Views (Login + MainWindow first, then all feature views in parallel)

Login and MainWindow are the shell that all feature views live inside. Build them first, then all 10 feature view pairs can be done simultaneously.

AdminSystem Views/ — Login + Shell first

✓	File	What to implement
☐	**Login.xaml / Login.xaml.cs**	XAML: centered card layout, TextBox for Email, PasswordBox for Password, Button (bound to LoginCommand), TextBlock for ErrorMessage (Visibility bound to BoolToVisibility converter). DataContext: LoginViewModel. Startup window.
☐	**MainWindow.xaml / MainWindow.xaml.cs**	XAML: DockPanel layout. Left sidebar: nav buttons (Dashboard, Orders, POS, Inventory, Products, Reports, Support, Users, Vouchers, Delivery, Logout). Each button command: NavigationHelper.NavigateTo<TargetView>(ContentFrame). Right: Frame/ContentControl named ContentFrame. Top status bar: logged-in user name and role. DataContext: DashboardViewModel.

AdminSystem Views/ — all feature views in parallel after MainWindow

✓	File	What to implement
☐	**Dashboard.xaml / Dashboard.xaml.cs**	XAML: 4 KPI tiles (TodaySales, PendingOrders, LowStockCount, OpenTickets) in WrapPanel. Binding to DashboardViewModel. Refresh button (LoadCommand). Display currency with CurrencyConverter.

✓	File	What to implement
☐	**Orders.xaml / Orders.xaml.cs**	XAML: top filter ComboBox (OrderStatus). DataGrid of orders (OrderNumber, Date, Status with StatusToColor, TotalAmount). Right panel: selected order detail (items DataGrid, payment info, pickup/delivery section). Buttons: UpdateStatus, SetPickupReady, ConfirmPickup. DataContext: OrderViewModel.
☐	**POS.xaml / POS.xaml.cs**	XAML: left panel: search box + product DataGrid. Centre: cart DataGrid (product, qty, price, remove). Bottom: totals (subtotal, discount, total). Payment section: Cash/GCash radio + amount. Voucher code input. Complete Transaction button. Print Receipt button. DataContext: POSViewModel. Most complex view — take care with layout.
☐	**Inventory.xaml / Inventory.xaml.cs**	XAML: TabControl — Tab 1: DataGrid of products with StockQuantity and ReorderThreshold columns (low stock rows highlighted red via DataTrigger). Adjust form: product selector, quantity input, type ComboBox, notes. Tab 2: InventoryLog DataGrid with date, type, quantity, who. DataContext: InventoryViewModel.
☐	**Products.xaml / Products.xaml.cs**	XAML: DataGrid of products. Right: edit form (Name, SKU, Price, Category ComboBox, Brand ComboBox, IsFeatured CheckBox). Variants sub-DataGrid (editable inline). Save/Cancel buttons. DataContext: ProductViewModel.
☐	**Reports.xaml / Reports.xaml.cs**	XAML: DatePicker range, ReportType ComboBox (Sales Summary, Top Products, Inventory). Generate button. Results: DataGrid for tabular data. WPF chart control (e.g. LiveCharts WPF) for revenue over time. Export to CSV button. DataContext: ReportViewModel.
☐	**Support.xaml / Support.xaml.cs**	XAML: left: ticket list DataGrid with status filter (StatusFilter ComboBox). Right panel: ticket details (category, subject, description, attachments link). Reply section: TextBox + Send. Tasks list DataGrid. Action buttons: Create Task (opens dialog), Resolve. DataContext: SupportViewModel.
☐	**Users.xaml / Users.xaml.cs**	XAML: DataGrid of users (Name, Email, Role badges, IsActive toggle). Right panel: role assignment checkboxes (Admin/Manager/Cashier/Staff/Customer). Enable/Disable toggle button. DataContext: UsersViewModel.
☐	**Vouchers.xaml / Vouchers.xaml.cs**	XAML: left: voucher list DataGrid (Code, Type, Value, Dates, IsActive). Right: create/edit form (Code input, DiscountType RadioButton, DiscountValue, MinimumAmount, MaxUses, MaxPerUser, DatePicker range). Save + Activate/Deactivate buttons. DataContext: VoucherViewModel.
☐	**Delivery.xaml / Delivery.xaml.cs**	XAML: DataGrid of deliveries (OrderNumber, Courier, Status with StatusToColor, Driver info if Lalamove, TrackingNumber if LBC, IsDelayed flag). Buttons: Dispatch, Mark Delayed (opens ETA dialog), Cancel. DataContext: DeliveryViewModel.

STEP 28 Remaining CSS and JavaScript

All remaining CSS and JS files for features built in Steps 12-20. All can be written in parallel with each other and with AdminSystem steps.

wwwroot/css/ — remaining (write all in parallel)

✓	File	What to implement
☐	**delivery-selection.css**	Delivery method radio cards: option card layout with courier logo placeholder, price display, selected state border highlight.
☐	**notification-alert.css**	Toast banner: fixed top-right position (or inline), success/error/info colour variants, dismiss button, fade-in animation.
☐	**order-summary.css**	Order totals panel: line items table, discount row (green text), shipping row, grand total (bold, larger font), sticky sidebar on desktop.
☐	**payment-method.css**	Payment method cards: radio button hidden, card clickable, selected state with border, icon+label layout, conditional content sections show/hide.
☐	**product-card.css**	Product card: image with hover zoom effect, 2-line title truncation, price bold, stock badge (Out of Stock / Low), button group at bottom.
☐	**product-catalog.css**	Catalogue: two-column layout (280px sidebar + grid). Grid: 2-4 cols depending on breakpoint. Filter sidebar sticky. Sort/search bar top.
☐	**product-details.css**	Product detail: image gallery (main + thumbnails row). Spec table. Variant selector radio cards. Add-to-cart section sticky bottom on mobile.

✓	File	What to implement
☐	**confirmation.css**	Success page: hero banner green, large checkmark icon, order number display, steps summary.
☐	**confirmation-modal.css**	Bootstrap modal: success animation (scale-in), order number large display, CTA button centered.
☐	**order-history.css**	Order cards: status badge colour map (use CSS custom properties matching StatusToColor values). Expandable detail row.

wwwroot/js/ — remaining (write all in parallel)

✓	File	What to implement
☐	**order-history.js**	Status filter tabs: data attribute filtering of order cards. Expand/collapse detail rows. Track polling: setInterval 10s AJAX GET OrderController.Track for active orders, update status text.
☐	**product-catalog.js**	Already specified in Step 12. Confirm: debounce search input (300ms), AJAX filter reload, wishlist toggle AJAX, variant price update.
☐	**confirmation.js**	DOMContentLoaded: if order success param present → show confetti (canvas-confetti from CDN or CSS animation). Auto-show _ConfirmationModal.
☐	**customer.js**	Already specified in Step 10. Confirm: profile dropdown, notification bell, session expiry warning.

STEP 29 End-to-End Testing & Postman Collections

Validate all 16 flowcharts work together as a complete system.

postman/ — write collection files in parallel

✓	File	What to implement
☐	**postman/collections/**	Create a Postman collection per major flow: Auth (register, verify OTP, login, logout), Catalog (list products, filter, get detail), Cart (add, update, remove, view), Checkout (review, address, delivery, create order), Payment (GCash, BankTransfer upload, status check), Orders (history, track, confirm delivery), Vouchers (validate code), Reviews (submit, get), Support (create ticket), Wishlist (toggle, view). Export as JSON to this folder.
☐	**postman/environments/**	Create: development.json (base_url=https://localhost:5001), staging.json. Include auth token variable that gets set by login request.

Manual flow verification checklist

✓	File	What to implement
☐	**Flowcharts 1-6 (Customer flows)**	Test: register with OTP, login, browse catalog with filters, add to cart as guest, login and merge cart, checkout with voucher, pay via GCash, pay via BankTransfer (upload proof), admin verify payment, track delivery, confirm receipt, submit review.
☐	**Flowcharts 7-13 (Admin flows)**	Test in AdminSystem: login, dashboard KPIs, process pickup order, verify bank transfer proof, dispatch Lalamove delivery, mark delivery delayed, process walk-in POS sale (Cash + GCash), adjust inventory, receive stock from supplier PO, reply to and resolve support ticket.
☐	**Flowcharts 14-16 (Automated + New)**	Test: background job fires on payment timeout (set deadline to 1 minute ago), stock alert fires when variant goes below threshold, wishlist restock notification sent. Voucher checkout: all 5 validation steps fail correctly, then valid code applies discount. Supplier PO: create, send, receive partial delivery, stock updates, InventoryLog entries correct.

Taurus Bike Shop · Implementation Checklist · 29 Steps · Schema v7.1 · 16 Flowcharts